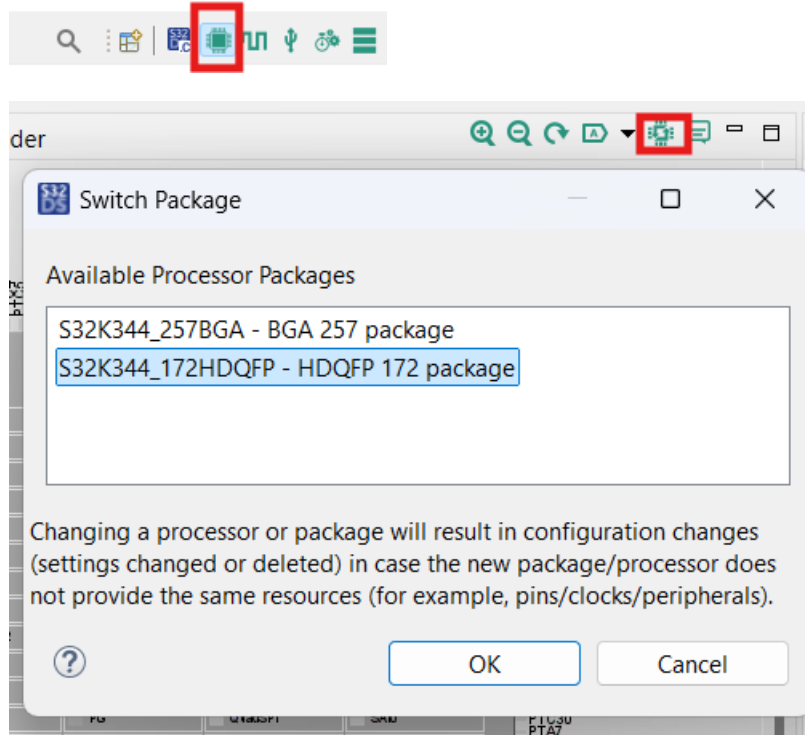


CONFIGURE TOOL

- 1. Select the package



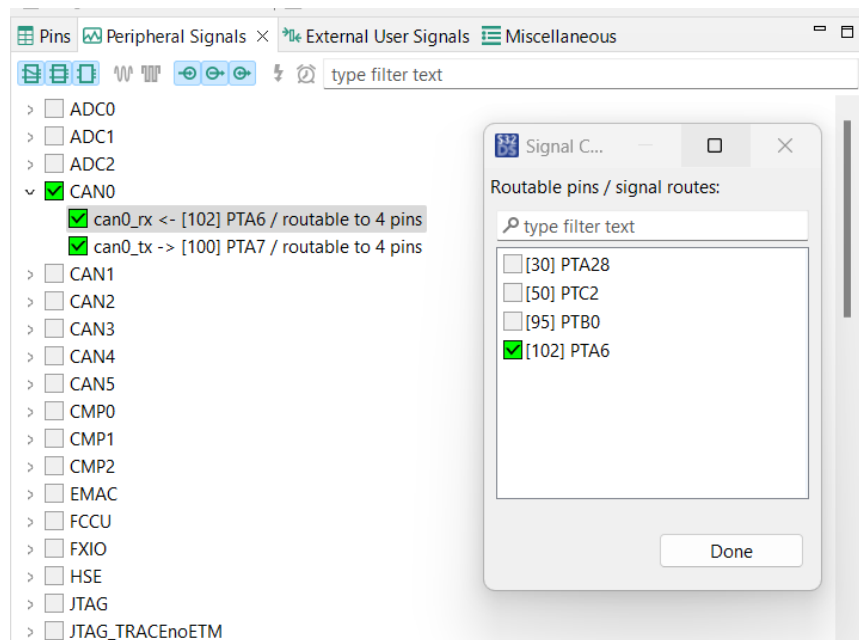
- 2. Add pins

- For Evaluation board S32K312EVb-T172, the following pin configurations are required:

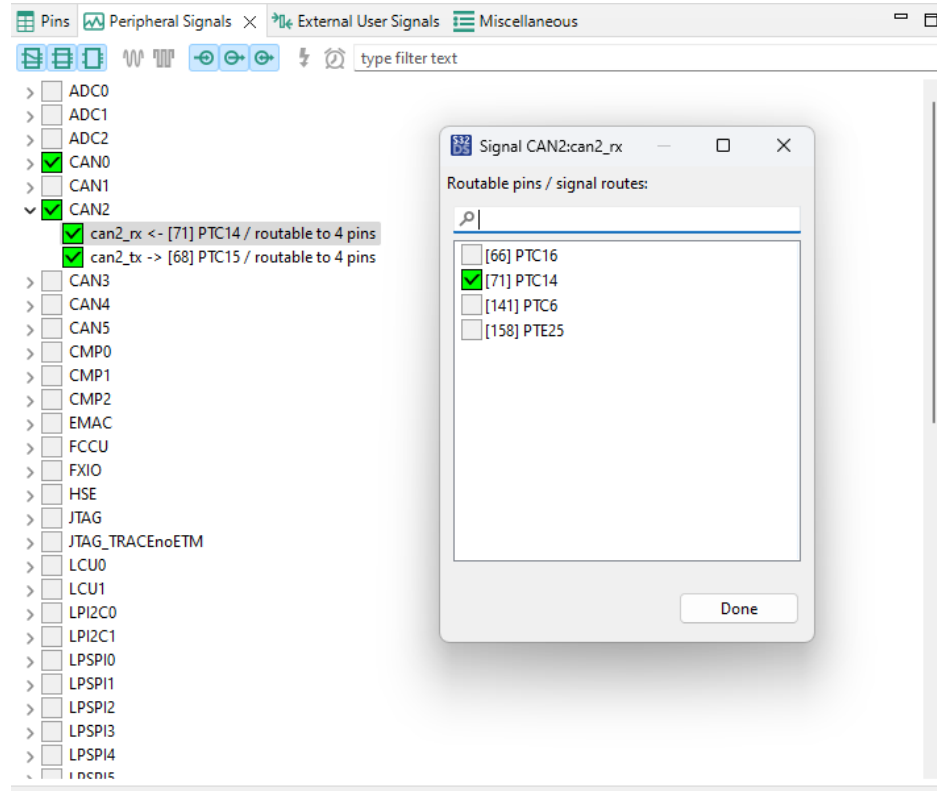
Table 12. CAN Interface – MCU Connections

S32K3X4EVb-T172 Board Version	TJA1153 Secure HS CAN	TJA1443 CAN FD	Signal Name	MCU Port	Comment/Description
Rev A	●		4 – CAN_RX	PTA6	[CAN0_RX Module] is routed to CAN Phy0
			1 – CAN_TX	PTA7	[CAN0_TX Module] is routed to CAN Phy0
Rev B			8 – NC	PTC23	In this PHY the pin is internally connected and should be floated
			6 – CAN_EN	PTC21	PTC21 is routed to CAN Phy0 as CAN0_EN
			14 – CAN_STB	PTC20	PTC20 is routed to CAN Phy0 as CAN0_STB
Rev B1 & B2		●	4 – CAN_RX	PTA6	[CAN0_RX Module] is routed to CAN Phy0
			1 – CAN_TX	PTA7	[CAN0_TX Module] is routed to CAN Phy0
			8 – CAN_ERRN	PTC23	PTC23 is routed to CAN Phy0 as CAN0_ERRN
			6 – CAN_EN	PTC21	PTC21 is routed to CAN Phy0 as CAN0_EN
			14 – CAN_STB	PTC20	PTC20 is routed to CAN Phy0 as CAN0_STB

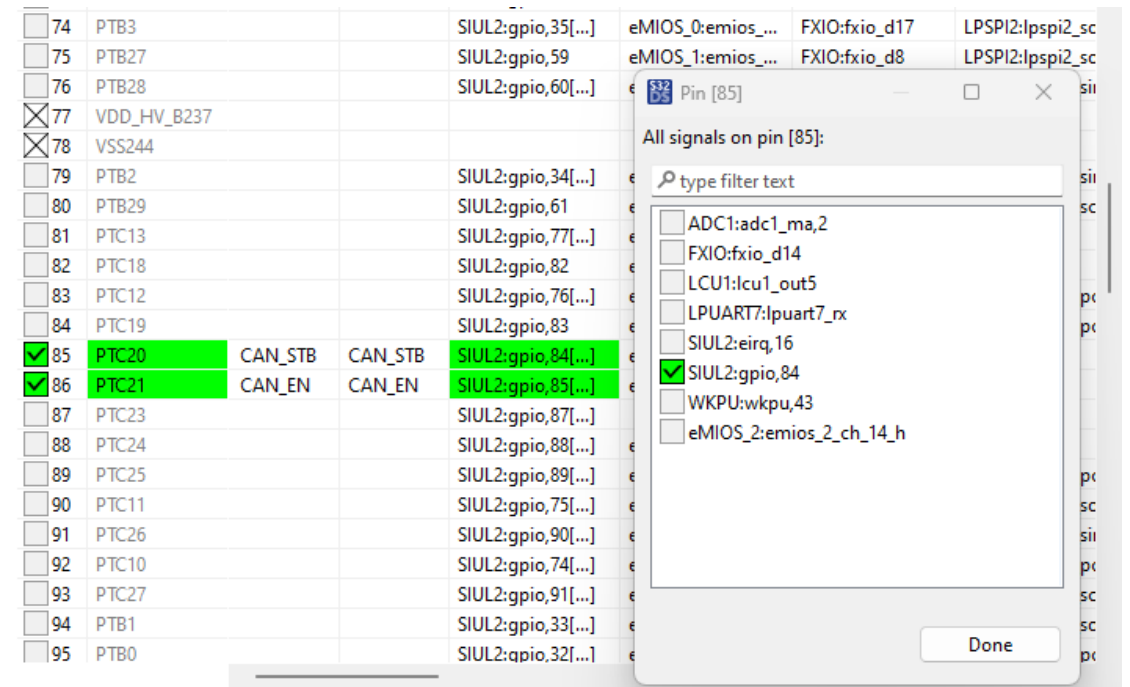
- CAN Channel 0 pins:



- CAN Channel 2 pins:**

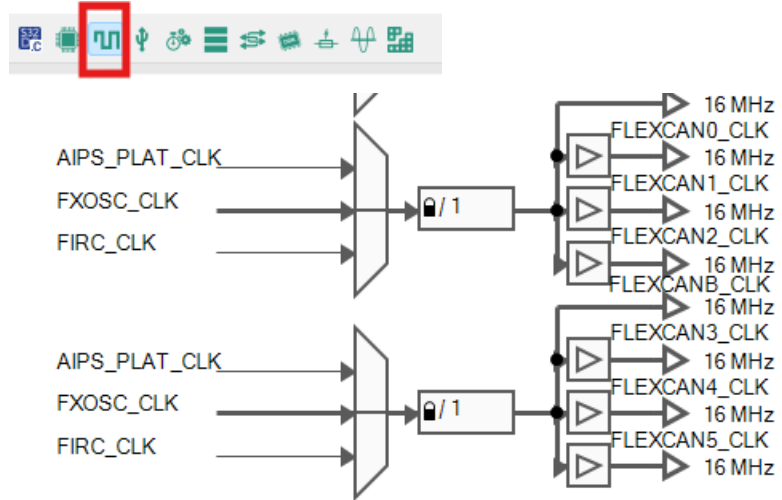


- CAN Transceiver pins (CAN_STB, CAN_EN):**

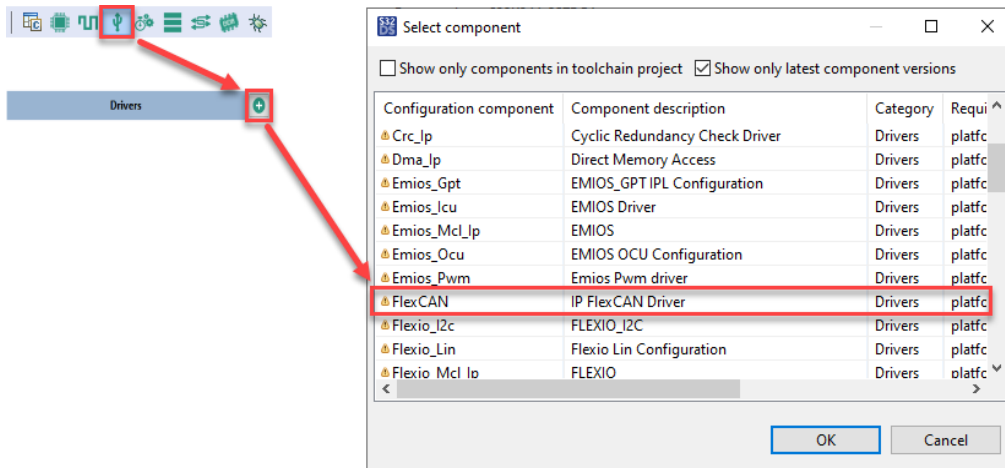


- 3. Configure the clocks

- CAN clock source will be configured to use the crystal clock populated on the EVB:



- 4. Add peripheral



CONFIGURE TOOL

- 1. Add/remove the used **can instance**: FlexCAN0 – 5.
- 2. Configure the **parameter** of module:
[Name]: will be the name of configure structure.
[Number Of MB] : 96(CAN0), 64(CAN1,2), 32(others), assuming 8 bytes payload.
[Payload Size] : 8, 16, 32, 64bytes.
[Rx FIFO Transfer Type] : interrupt, polling, DMA.

[FlexCAN Protocol Clock] : Input the clock number in FlexCAN module.

- 3. Configure **bit rate**
CAN arbitration phase bit rate. Note the value which fill in here is the exact value in corresponding segment, register value will subtract 1.

- Configure **data bit rate**
CAN data phase bit rate.

IP FlexCAN Driver [Drivers] 2

Name FlexCAN Custom name ☐

Mode General Mode

Preset Custom...

Name Config Time Support FlexCAN configurations FlexCAN General Timeout Configuration

+ x ^ v

0
1

FlexCAN Hardware Channel FLEXCAN_0

Name FlexCAN_Config0

Number Of MB 32

FlexCAN Rx FIFO filters number 8 Rx FIFO Filters

Enable Legacy Rx FIFO ☐

Enable FD Can ☐

FlexCAN operation modes Normal mode or user mode

FlexCAN Ram Block

Choice FlexCAN Ram Block Unified

FlexCAN Ram Block Unified

Ram Block Unified Payload Length FLEXCAN_PAYLOAD_SIZE_8

Rx FIFO Transfer Type Using interrupts

DMA Channel Used 0

Enable EnhancedCBT Can ☐

Enable BitRate Switch ☐

Can FD ISO ☐

Auto Bus Off ☒

Remote Request Store ☐

Lowest Buffer Transmitted First ☐

TimeStamp Free Running Timer Source Captured time base is CAN bit clock.

3 TimeStamp Timer Source Message buffer time stamp base is TIMER.

TimeStamp HR Capture Config The high resolution time stamp capture is disabled. Enable Timer Source

FlexCAN Protocol Clock 16000000

FlexCAN bitrate

Name FlexCAN_Ip_TimSeg0

CONFIGURE TOOL

- Configure **data bit rate**
CAN data phase bit rate. Some limitation must be followed in protocol or reference manual. The minimum number of time quanta per CAN bit must be 8.

- Callback function** and enhanced **FIFO** setting
Enable or disable enhanced FIFO, configure the filter table elements number, extended filter elements number and watermark of FIFO engine.

Click 'Update Code' to generate code after configuration

The screenshot shows the 'FlexCAN bitrate' and 'FlexCAN CBT bitrate' configuration sections. An arrow points to the 'FlexCAN bitrate' section, and another arrow points to the 'FlexCAN CBT bitrate' section. A third arrow points to the 'Callback' and 'Error Callback' section.

FlexCAN bitrate	
Name	FlexCAN_Ip_TimeSeg0
Synchronization segment	1
Propagation segment	2
Phase segment 1	3
Phase segment 2	2
Prescaler division factor	4
Resync jump width	2
Bitrate (Kbps)	500
Sampling point (%)	75

FlexCAN CBT bitrate	
Name	FlexCAN_Ip_TimeSegCBT0
Synchronization segment	1
Propagation segment	7
Phase segment 1	6
Phase segment 2	2
Prescaler division factor	5
Resync jump width	2
Bitrate (Kbps)	200
Sampling point (%)	87.5

Callback	
Callback	Can0_MB_RXTC_CALLBACK
Error Callback	Can0_Error_Callback
The number of enhanced ID filter elements	0
The number of enhanced extended ID filter elements	0
The number of enhanced Rx FIFO watermarks/The number of Message Buffers Transferred by DMA	0
Enable/Disable the Enhanced Rx FIFO feature	☐

The screenshot shows the bottom toolbar with a dropdown menu set to 'S32K3_RTD_Training_CAN', a home icon, a warning icon, and a red-bordered 'Update Code' button.

GENERATED CODE

• RTD

include

- > .h FlexCAN_Ip_DeviceReg.h
- > .h FlexCAN_Ip_HwAccess.h
- > .h FlexCAN_Ip_Irq.h
- > .h FlexCAN_Ip_TrustedFunctions.h
- > .h FlexCAN_Ip_Types.h
- > .h FlexCAN_Ip_Wrapper.h
- > .h FlexCAN_Ip.h

src

- > .c FlexCAN_Ip_HwAccess.c
- > .c FlexCAN_Ip_Irq.c
- > .c FlexCAN_Ip.c

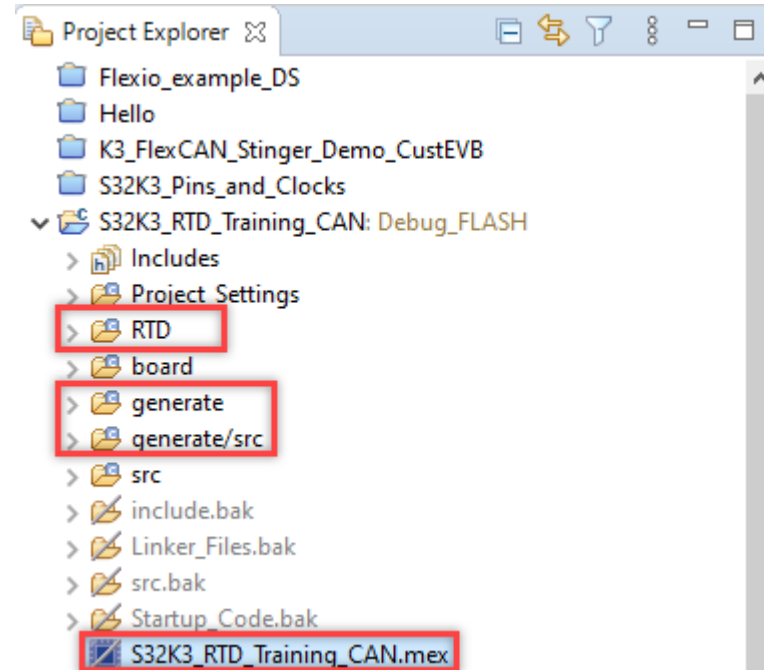
generate

include

- > .h Clock_Ip_Cfg_Defines.h
- > .h FlexCAN_Ip_Cfg.h
- > .h FlexCAN_Ip_CfgDefines.h
- > .h FlexCAN_Ip_Sa_BOARD_InitPeripherals_PBcfg.h

generate/src

- > .c FlexCAN_Ip_Sa_BOARD_InitPeripherals_PBcfg.c



```
/* FlexCAN State Structure used By driver
User should not modify this structure */
Flexcan_Ip_StateType FlexCAN_State0;
/* FlexCAN State Structure used By driver
User should not modify this structure */
Flexcan_Ip_StateType FlexCAN_State1;
```

```
const Flexcan_Ip_ConfigType FlexCAN_Config4 = {
    /* Number Of Message Buffer used .max_num_mb */
    (uint8)32,
    /*Can Hw filter count* .num id filters*- aici exista
    variatna sa generezi toate filtrele si sa referentiezi tu
    in cod */
    FLEXCAN_RX_FIFO_ID_FILTERS_8,
    /* Legacy FIFO ENABLED .is_rx_fifo_needed*/
    (boolean)FALSE,
    #if (FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO == STD_ON)
    /* The number of standard ID filter elements */
    0U,
    /* The number of extended ID filter elements */
    0U,
    /* The number of enhanced Rx FIFO watermark */
    0U,
    /* The Enhanced Rx FIFO feature is enabled or not. */
    (boolean)FALSE,
    #endif
    FLEXCAN_NORMAL_MODE,
```

GENERATED CODE

• FLEXCAN structure – in FlexCAN_Ip_Types.h

1

```
typedef struct
{
    uint32 max_num_mb;

    Flexcan_Ip_RxFifoIdFilterNumType num_id_filters;

    boolean is_rx_fifo_needed;

    #if (FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO == STD_ON)
    uint32 num_enhanced_std_id_filters;

    uint32 num_enhanced_ext_id_filters;

    uint32 num_enhanced_watermark;

    boolean is_enhanced_rx_fifo_needed;

    #endif /* FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO */
    #if (FLEXCAN_IP_FEATURE_HAS_TS_ENABLE == STD_ON)
    Flexcan_Ip_TimeStampConfigType time_stamp;
    #endif /* FLEXCAN_IP_FEATURE_HAS_TS_ENABLE */
    Flexcan_Ip_ModesType flexcanMode;

    uint32 ctrlOptions;
    Flexcan_Ip_PayloadSizeType payload;
    boolean fd_enable;
    #if (FLEXCAN_IP_FEATURE_HAS_PE_CLKSRC_SELECT == STD_ON)
    boolean is_pe_clock;
    #endif

    boolean enhCbtEnable;
    boolean bitRateSwitch;
    Flexcan_Ip_TimeSegmentType bitrate;
    Flexcan_Ip_TimeSegmentType bitrate_cbt;
    Flexcan_Ip_RxFifoTransferType transfer_type;
    #if (FLEXCAN_IP_FEATURE_HAS_DMA_ENABLE == STD_ON)
    uint8 rxFifoDMAChannel;
    #endif /* FLEXCAN_IP_FEATURE_HAS_DMA_ENABLE */
    FlexCAN_Ip_CallbackType Callback;
    FlexCAN_Ip_ErrorCallbackType ErrorCallback;
} Flexcan_Ip_ConfigType;
```

2

```
typedef struct FlexCANState
{
    Flexcan_Ip_MBhandleType mbs[FLEXCAN_IP_FEATURE_MAX_MB_NUM];

    #if (FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO == STD_ON)
    Flexcan_Ip_MBhandleType enhancedFifoOutput;
    #endif /* FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO */
    void (*callback)(uint8 instance,
                    Flexcan_Ip_EventType eventType,
                    uint32 buffIdx,
                    const struct FlexCANState *driverState
                    );
    void *callbackParam;

    void (*error_callback)(uint8 instance,
                          Flexcan_Ip_EventType eventType,
                          uint32 u32ErrStatus,
                          const struct FlexCANState *driverState
                          ); /*< Error IRQ handler callback
    void *errorCallbackParam;

    #if (FLEXCAN_IP_FEATURE_HAS_DMA_ENABLE == STD_ON)
    uint8 rxFifoDMAChannel;
    #if (FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO == STD_ON)
    uint32 u32NumOfMbTransferByDMA;
    #endif /* (FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO == STD_ON) */
    #endif /* (FLEXCAN_IP_FEATURE_HAS_DMA_ENABLE == STD_ON) */
    Flexcan_Ip_RxFifoTransferType transferType;
    boolean bIsLegacyFifoEn;
    #if (FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO == STD_ON)
    boolean bIsEnhancedFifoEn;
    #endif /* (FLEXCAN_IP_FEATURE_HAS_ENHANCED_RX_FIFO == STD_ON) */
    uint32 u32MaxMbNum;
    boolean isIntActive;
} Flexcan_Ip_StateType;
```

```
typedef struct
{
    Flexcan_Ip_MsgBuffType * pMBmessage;
    volatile Flexcan_Ip_MbStateType state;
    #if FLEXCAN_IP_FEATURE_HAS_DMA_ENABLE
    volatile boolean isDmaBusy;
    #endif /* FLEXCAN_IP_FEATURE_HAS_DMA_ENABLE
    boolean isPolling;
    boolean isRemote;
    uint32 time_stamp;
} Flexcan_Ip_MBhandleType;

typedef struct
{
    uint32 cs;
    uint32 msgId;
    uint8 data[64];
    uint8 datalen;
    uint8 id_hit;
    uint32 time_stamp;
} Flexcan_Ip_MsgBuffType;
```

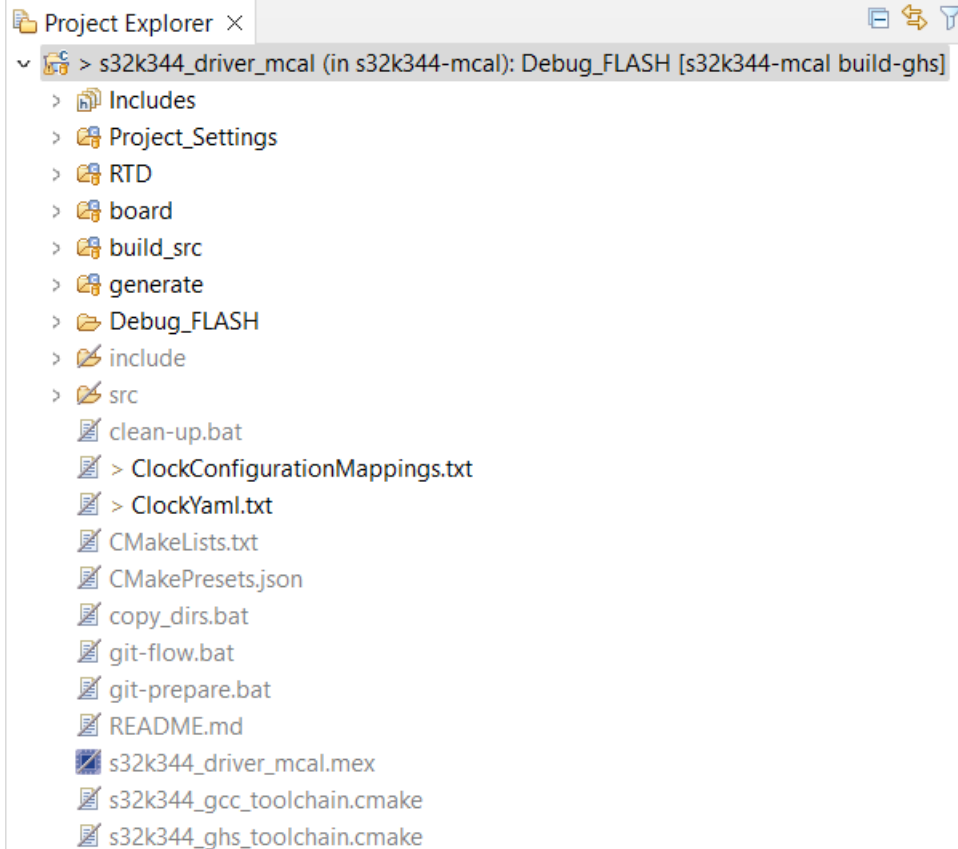
Three mb state :

FLEXCAN_MB_IDLE,
FLEXCAN_MB_RX_BUSY
FLEXCAN_MB_TX_BUSY

S32DS Test build

- Go back to S32 C/C++ Project

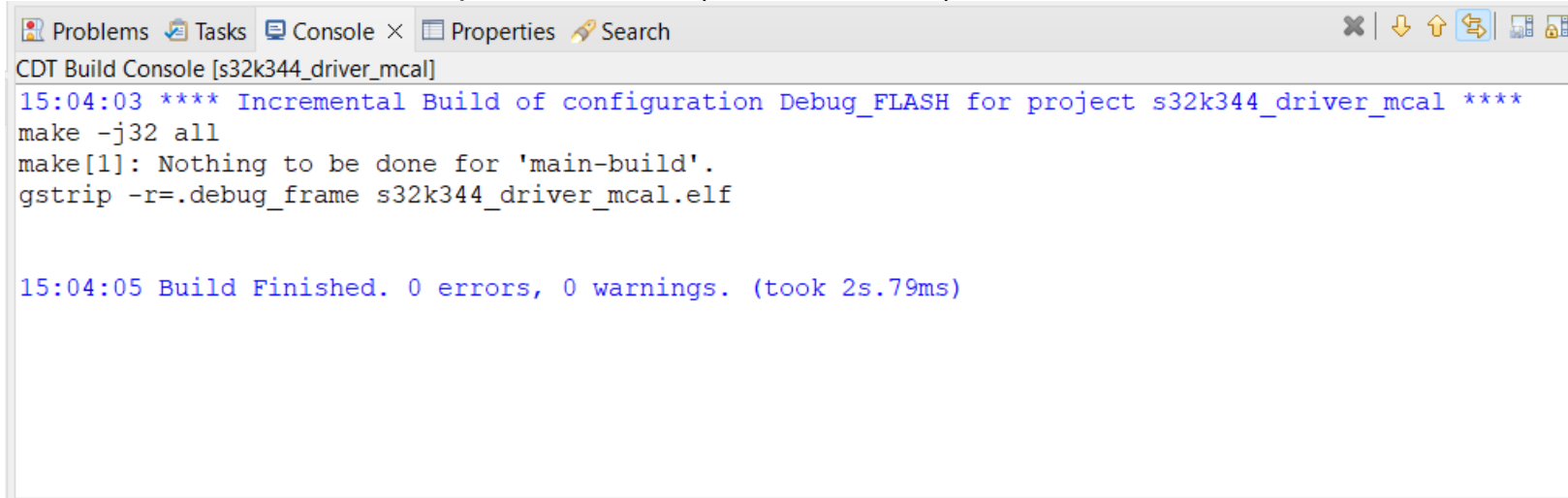
- 
- Right Click on the actual Project Explorer folder:



-
- Click Clean project
- Then, build the project:



- Make sure there are no compilation issues (on the console):



The screenshot shows the CDT Build Console window for the project 's32k344_driver_mcal'. The window has a title bar with tabs for 'Problems', 'Tasks', 'Console', 'Properties', and a 'Search' icon. The console output is as follows:

```
CDT Build Console [s32k344_driver_mcal]
15:04:03 **** Incremental Build of configuration Debug_FLASH for project s32k344_driver_mcal ****
make -j32 all
make[1]: Nothing to be done for 'main-build'.
gstrip -r=.debug_frame s32k344_driver_mcal.elf

15:04:05 Build Finished. 0 errors, 0 warnings. (took 2s.79ms)
```